Model-Guided Rocket Configuration Optimization Under a Cache Capacity Budget

Rohan Muthyala
Virginia Tech
rohan47@vt.edu

Shiva Pandeti
Virginia Tech
shivp@vt.edu

Tharakeshvar Padmanaphan Anandaraj
Virginia Tech
tharakeshvar@vt.edu

***Abstract*—Project category: OPT. Designers often tune a processor one setting at a time and hope the gains add up. We test that assumption on the Rocket core. We change seven settings (cache geometry, replacement policy, L2 size and BTB size), run six programs, and record execution cycles. From the one-at-a-time results we predict the cycles of combined configurations with an additive and a multiplicative model, then build a few of the predicted candidates to see how close the predictions are. The goal is the configuration with the best geometric-mean speedup whose cache data capacity does not exceed the baseline and that slows no program by more than 5%. The chosen configurations are re-run on a second input that was not used for selection. The study uses at most 12 configurations and 90 planned runs. We do not propose new hardware or claim a global optimum.**

I. Introduction

Processor settings often interact. Cache geometry and replacement policy, for example, together decide which data stay in the cache, and a configuration that suits one program can hurt another. This project builds on our labs. In HW3 we swept L1D sets and ways on matrix multiplication, in HW4 we compared random, LRU and pseudo-LRU replacement on the same kernel, in HW6 we compared CPI on Rocket, SmallBoom and LargeBoom, and in HW7 we compared branch predictors on SmallBoom. Those labs vary one setting on one program. We ask whether single-setting measurements like these can be combined to pick one shared configuration for six programs under a capacity limit and a limit on any single slowdown.

Our questions are: (1) How sensitive is each program to each setting? (2) How close do predictions from single-setting results come to measured combinations? (3) Does a configuration chosen on one input still help on another? If the answers are favorable, a designer can screen with a handful of builds. If not, we will show where interactions or input changes break the shortcut.

II. Related Work

Chipyard [1] is the framework we build and simulate with, and it generates Rocket cores from the Rocket Chip generator [2]. BOOM-Explorer [3] searches the BOOM design space with Bayesian optimization, trading power against cycles. We work on a much smaller Rocket space, use only cycle counts, and rely on two simple formulas. We do not try to reproduce BOOM-Explorer or beat it.

III. Optimization Objective

We maximize the geometric-mean speedup over six applications, relative to the baseline, subject to two constraints: (i) the combined data capacity of the L1I, L1D and L2 caches is at most the baseline's, and (ii) no application takes more than $1.05\times$ its baseline cycles. Capacity counts data arrays only. Tags, replacement state and BTB storage are left out, so this is not total storage, area or power.

IV. Proposed Method

Platform and parameters. We use one Rocket core from the course Chipyard setup, simulated with Verilator. The baseline is `ECE5504RocketL2Config` at 11026bca: L1I 64 sets $\times$ 1 way, L1D 8 sets $\times$ 8 ways, 64-byte lines and a 64 KiB 8-way inclusive L2, for a baseline cache data capacity of 72 KiB (4 + 4 + 64). The ISA, compiler options, line size, memory latency and everything else outside the study stay fixed.

Parameter	Baseline	Alternative	Capacity (KiB)
L1D sets	8	32	grows, 4→16
L1D ways	8	4	shrinks, 4→2
L1D replacement	random	pseudo-LRU	unchanged
L1I sets	64	32	shrinks, 4→2
L1I ways	1	2	grows, 4→8
L2 capacity	64	32	shrinks, 64→32
BTB entries	28	14	not counted

Every alternative must differ from its baseline and build. Some alternatives shrink capacity and others grow it, so trades within the budget exist, for example a 32 KiB L2 paired with a 16 KiB L1D. The L2 is 64 of the 72 KiB, so we expect most trades to be L2 against L1. The L2 size is set by the capacity argument of the inclusive cache, and we confirm in week 1 that 32 KiB builds; if it does not, we substitute L2 ways. In HW4 the replacement policy was changed by editing `HellaCache.scala`, so we will turn it into a config option or build it in a separate checkout; we also confirm in week 1 that the BTB-size change builds.

Screening and prediction. We build the baseline and seven configurations that each change one setting. These may exceed the budget, since they only measure isolated effects, but only budget-compliant configurations can be selected. For one

application, let C_0 be baseline cycles and C_i the cycles when only setting i changes. For a combination of changes:

$$\text{Additive: } \hat{C} = C_0 + \textstyle\sum_i (C_i - C_0),$$
$$\text{Multiplicative: } \hat{C} = C_0 \textstyle\prod_i (C_i / C_0).$$

We list all combinations of the tested levels, compute each one's capacity, and drop those over budget. Each model then ranks what is left by predicted geometric-mean speedup, requiring $\hat{C} \leq 1.05\, C_0$ for every application. Nonpositive predictions are discarded.

Combined configurations. Each model's top-ranked untested combination gets a slot, and a pick shared by both models counts once. The remaining slots go to distinct interaction tests, such as L1D sets with ways, or geometry with replacement. We record these choices before seeing results and build at most four, for twelve configurations in total. The baseline stays eligible as a fallback.

V. Evaluation Plan

Applications. We use matrix multiplication, quicksort and Dhrystone from our HW3–HW6 labs, and write or adapt three more: streaming vector addition, strided array summation and linked-list traversal. We document sources, working sets, strides, input generation and correctness checks, and the list uses a fixed, shuffled order. Memory-focused inputs are larger than the cache sizes under test. Dhrystone is a mostly cache-resident control, and we measure its sensitivity instead of assuming it. We freeze selection inputs after the early diagnostic and use identical binaries and timing boundaries across configurations.

Metrics and hypotheses. We report execution cycles and cache data capacity: speedup C_0/C per application, its geometric mean, and the worst slowdown, with CPI and cache counters to explain results. We assume an unchanged clock period and make no timing, power or energy claims. Prediction error is $100\,|\hat{C} - C_{\text{meas}}|/C_{\text{meas}}$. For a pair of settings, interaction is $C_{AB} - C_A - C_B + C_0$; a negative value means fewer cycles than the additive model predicts. We expect that (H1) L1D geometry and replacement interact, so additive predictions err most on combinations containing them; (H2) the streaming and strided programs respond mainly to L1D geometry, and the linked list also to L2 size; (H3) configuration rankings change with working-set size. We will report results that contradict these, and our conclusions cover only the combinations we tested.

Validation and run budget. Once selection is frozen, we run up to three configurations on a second input per application that played no part in tuning: the baseline, the best measured configuration meeting both constraints, and the best measured configuration within the capacity budget that ignores the slowdown limit (identical picks run once). Validation never triggers reselection. Twelve configurations on six applications give at most 72 selection runs, plus at most 18 validation runs: 90 planned runs in all. Failed builds and debugging runs are logged separately.

VI. Timeline and Contributions

Nov 1–7	Verify settings and L2 support; baseline correctness; time a build and a run; early diagnostic with one smaller-L1D configuration
Nov 8–14	Finish the seven single-setting runs; compute predictions
Nov 15–21	Run up to four combined configurations; freeze selections
Nov 22–28	Validation, interaction analysis, figures
Nov 29–Dec 1	Recheck results; presentation

If the memory-focused programs barely react in the early diagnostic, we check working-set sizes, access patterns and instrumentation before freezing inputs; validation inputs stay separate. Rohan leads L1D configuration and the experiment scripts, Shiva leads L1I and L2 configuration and the predictions, and Tharakeshvar leads BTB configuration, the interaction analysis and the figures. Each of us owns two applications and helps with analysis and writing, and all of us commit regularly and keep the weekly log. We budget five hours per person per week. If runs take longer than expected, we cut combined configurations first and keep all seven settings and six programs. The final report is due at noon on Dec. 16.

AI Usage Statement

We used AI tools (Claude and ChatGPT) to rephrase and enhance the content of this proposal, to generate the LaTeX code, and to analyze the feasibility of the plan and schedule. The AI did not run any experiments, and no results are reported.

References

[1] A. Amid *et al.*, "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[2] K. Asanović *et al.*, "The Rocket Chip generator," Univ. California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016.

[3] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. F. Wong, "BOOM-Explorer: RISC-V BOOM microarchitecture design space exploration framework," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2021, doi: 10.1109/ICCAD51958.2021.9643455.